

KNOWTIFY

Guion de Exposición

Fase de Diseño · Backend · Frontend

Dividido en 3 integrantes

Integrante 1 — Fase de Diseño

Integrante 2 — Backend

Integrante 3 — Frontend

Cómo usar este guion

- Cada integrante tiene su propia sección con: lo que debe DECIR (viñetas en lenguaje hablado), QUÉ MOSTRAR en pantalla y los TÉRMINOS CLAVE que debe dominar.
- Las viñetas no son para leerlas textualmente: son una guía. Habla con naturalidad y mira al público.
- Tiempo total sugerido: 10–12 minutos (3–4 minutos por integrante) + preguntas.
- Antes de empezar: tener la app abierta en el navegador (<http://localhost:3000>) y el Swagger en <http://localhost:8000/docs>.
- Al final hay una sección de POSIBLES PREGUNTAS DEL JURADO con respuestas para los tres.

Resumen de la división:

Integrante	Tema	Idea central
Integrante 1	Fase de Diseño	Cómo se pensó el sistema antes de programarlo: arquitectura, datos, interfaz y el motor OMR.
Integrante 2	Backend	El 'cerebro': API, base de datos, calificación con visión por computador, IA y seguridad.
Integrante 3	Frontend	Lo que ve el usuario: portales del docente y del estudiante, diseño y experiencia.

Integrante 1 — Fase de Diseño

Tiempo sugerido: 3–4 minutos. Eres quien ABRE la exposición.

1. Presentación e introducción del problema

Qué decir:

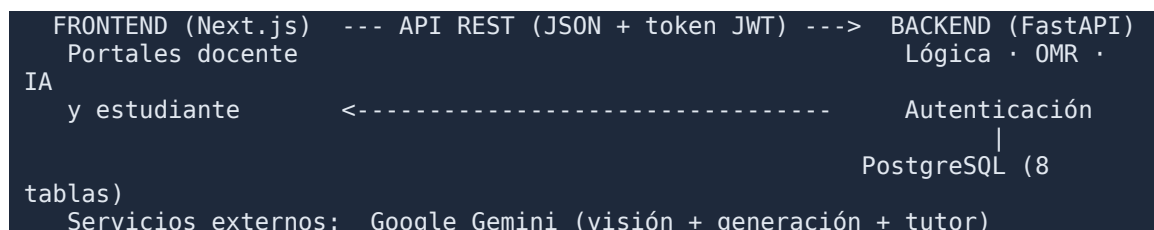
- Buenas, somos el equipo de KNOWTIFY, una plataforma educativa de calificación inteligente.
- El problema: calificar exámenes consume muchísimo tiempo del docente, y al estudiante la información de su desempeño le llega tarde y poco clara.
- Nuestra solución reúne en una sola herramienta tres cosas: calificar exámenes automáticamente con visión por computador e IA, gestionar las notas por periodo, y comunicar el desempeño al estudiante.
- Yo les voy a explicar la FASE DE DISEÑO: cómo pensamos el sistema antes de programarlo.

2. Metodología y arquitectura

Qué decir:

- Trabajamos con una metodología incremental: construimos por partes funcionales y verificables, sobre una base siempre estable.
- Diseñamos una arquitectura de tres capas separadas: la interfaz (frontend), la lógica (backend) y la base de datos. Se comunican por una API REST con formato JSON.
- Esta separación permite que la parte visual y el servidor evolucionen por separado, y que la información sensible viva siempre en el servidor.

Mostrar en pantalla — Diagrama de arquitectura (puedes leerlo así):



3. Diseño del modelo de datos

Qué decir:

- Diseñamos 8 entidades (tablas). Las principales: Estudiantes, Usuarios (docentes), y Notas.

- Una decisión importante: las notas se guardan por las tres dimensiones Saber, Hacer y Ser, y por periodo; el sistema calcula la nota final y el nivel de desempeño.

```
1 class Student(Base):
2     __tablename__ = "students"
3     id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
4     full_name = Column(String, nullable=False)
5     document_id = Column(String, unique=True, index=True)
6     grade = Column(String, default="11", index=True)
7     grupo = Column(String, default="1", index=True)
8
9 class Grade(Base):                                # notas por dimensión y periodo
10     __tablename__ = "grades"
11     period_id = Column(Integer, nullable=False, index=True)
12     score_saber = Column(Numeric(3, 2), default=1.0)
13     score_hacer = Column(Numeric(3, 2), default=1.0)
14     score_ser = Column(Numeric(3, 2), default=1.0)
15     final_period_score = Column(Numeric(3, 2))
16     performance_level = Column(String) # SUPERIOR | ALTO | BÁSICO | BAJO
```

*Mostrar en pantalla — Modelo de datos (entidades Estudiante y Notas) · archivo:
04_codigo_backend/04_modelado_postgresql.png*

4. Diseño de la interfaz y del motor OMR

Qué decir:

- En la interfaz diseñamos DOS portales diferenciados por color: docente (índigo) y estudiante (violeta), con modo claro/oscuro y ayuda contextual.
- Para calificar diseñamos una hoja de respuestas con cuatro cuadros en las esquinas (fiduciales) que permiten corregir la foto y leerla siempre igual.
- Decisión clave de diseño: en vez de pedir el número de documento, el sistema identifica al estudiante leyendo su NOMBRE escrito, con ayuda de la IA. Cero pérdida de tiempo.

KNOWTIFY – Hoja de respuestas

NOMBRE Y APELLIDOS (escribe en letra clara):

RESPUESTAS

- | | |
|--|--|
| 1 ☐ A ☐ B ☐ C ☐ D | 16 ☐ A ☐ B ☐ C ☐ D |
| 2 ☐ A ☐ B ☐ C ☐ D | 17 ☐ A ☐ B ☐ C ☐ D |
| 3 ☐ A ☐ B ☐ C ☐ D | 18 ☐ A ☐ B ☐ C ☐ D |
| 4 ☐ A ☐ B ☐ C ☐ D | 19 ☐ A ☐ B ☐ C ☐ D |
| 5 ☐ A ☐ B ☐ C ☐ D | 20 ☐ A ☐ B ☐ C ☐ D |
| 6 ☐ A ☐ B ☐ C ☐ D | 21 ☐ A ☐ B ☐ C ☐ D |
| 7 ☐ A ☐ B ☐ C ☐ D | 22 ☐ A ☐ B ☐ C ☐ D |
| 8 ☐ A ☐ B ☐ C ☐ D | 23 ☐ A ☐ B ☐ C ☐ D |
| 9 ☐ A ☐ B ☐ C ☐ D | 24 ☐ A ☐ B ☐ C ☐ D |
| 10 ☐ A ☐ B ☐ C ☐ D | 25 ☐ A ☐ B ☐ C ☐ D |
| 11 ☐ A ☐ B ☐ C ☐ D | |
| 12 ☐ A ☐ B ☐ C ☐ D | |
| 13 ☐ A ☐ B ☐ C ☐ D | |
| 14 ☐ A ☐ B ☐ C ☐ D | |
| 15 ☐ A ☐ B ☐ C ☐ D | |

Rellena completamente la burbuja. Escribe tu nombre completo arriba.

*Mostrar en pantalla — Hoja de respuestas diseñada (fiduciales, nombre y burbujas) · archivo:
06_omr/01_hoja_respuestas.png*

Cierre de tu parte:

- Con el diseño definido, le paso la palabra a [Integrante 2], que les explica cómo se construyó el backend.

Integrante 2 — Backend

Tiempo sugerido: 3–4 minutos. Eres el 'motor' técnico.

1. Stack y API REST

Qué decir:

- El backend es el cerebro del sistema. Lo construimos con FastAPI (Python), SQLAlchemy para la base de datos PostgreSQL, y Pydantic para validar los datos.
- Expone alrededor de 30 endpoints REST. Lo bueno de FastAPI es que genera SOLA la documentación interactiva, donde se pueden probar todos los servicios.

MOCABI EDU-GRADE API 1.0.0 OAS 3.1
/openapi.json

			Authorize
default ^			
POST	/login/teacher	Login Teacher	▼
POST	/login/student	Login Student	▼
POST	/newsletter	Newsletter Subscribe	▼
GET	/newsletter/subscribers	Newsletter List	🔒 ▼
GET	/student/dashboard	Student Dashboard	🔒 ▼
GET	/students/	Get Students	🔒 ▼
POST	/students/	Create Student	🔒 ▼
POST	/students/bulk	Bulk Import Students	🔒 ▼
DELETE	/students/{student_id}	Delete Student	🔒 ▼
GET	/grades/	Get Grades	🔒 ▼
POST	/grades/	Create Or Update Grade	🔒 ▼
POST	/upload-exams	Upload Exams	🔒 ▼

Mostrar en pantalla — Documentación interactiva de la API (Swagger) · archivo:
05_api/01_swagger_documentacion.png

2. Motor de calificación OMR (visión por computador)

Qué decir:

- Usamos OpenCV para leer las hojas. El motor corrige la perspectiva de la foto, mide cuánto está rellena cada burbuja A/B/C/D y la compara con la clave de respuestas.

- Funciona con fotos y con PDF, en milisegundos por hoja, sin internet y sin coste por hoja.
- La nota se calcula en escala de 1.0 a 5.0.

```

1 # La MISMA geometría genera la plantilla imprimible y lee las marcas
2 SHEET_W, SHEET_H = 1000, 1414 # lienzo normalizado (A4)
3 GRID_TOP, ROW_H, BUBBLE_R = 380, 50, 16
4
5 def read_marks(file_bytes, content_type, num_q, num_opts=4):
6     pages = _load_pages(file_bytes, content_type) # PDF o imagen
7     for gray in pages:
8         warped, aligned = _warp_to_canvas(gray) # corrige perspectiva
9         for q, opts in bubble_centers(num_q).items():
10             ratios = [_fill_ratio(bin_img, cx, cy, BUBBLE_R) for cx, cy in opts]
11             answers[q] = OPTION_LETTERS[int(np.argmax(ratios))]
12
13 # Comparación con la clave (JSON) y nota en escala 1.0 - 5.0
14 is_ok = sel is not None and sel == key_norm.get(q)
15 score = round(correct / num_q * 4 + 1, 2)

```

Mostrar en pantalla — Lectura de marcas con OpenCV y cálculo de la nota · archivo: 04_codigo_backend/03_calificacion_masiva_omr.png

3. Inteligencia Artificial (Gemini)

Qué decir:

- Integramos Google Gemini para tres cosas: identificar el nombre escrito en cada hoja, generar exámenes a partir de un tema, y el tutor virtual del estudiante.
- Para identificar al estudiante, recortamos el nombre y se lo enviamos a la IA junto con la lista de la clase; el sistema normaliza tildes y mayúsculas para emparejar bien.

```

1 client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
2
3 def get_active_model_name() -> str:
4     for m in client.models.list():
5         if "generateContent" in m.supported_actions and "flash" in m.name:
6             return m.name # auto-detecta el modelo Flash disponible
7     return "gemini-2.0-flash" # fallback
8
9 MODEL_NAME = get_active_model_name()
10
11 # Lectura de nombres manuscritos: se envían los recortes + la lista de la clase
12 contents = []
13 for n, (_, i, img) in enumerate(valid, start=1):
14     contents.append(f"Hoja {n}:")
15     contents.append(types.Part.from_bytes(data=img, mime_type="image/png"))
16 resp = await asyncio.to_thread(client.models.generate_content,
17                                model=MODEL_NAME, contents=contents)

```

Mostrar en pantalla — Integración con Gemini (visión) · archivo: 04_codigo_backend/01_integracion_gemini.png

4. Seguridad y tiempo real

Qué decir:

- La seguridad: autenticación con tokens JWT, contraseñas cifradas con bcrypt, validación por roles (docente/estudiante) y límite de intentos para evitar abusos.
- Para la mensajería usamos WebSockets, que permiten enviar los mensajes en tiempo real, con un respaldo automático por si la conexión falla.

```
1 def get_current_user(credentials = Depends(security)) -> dict:
2     payload = verify_token(credentials.credentials) # valida y decodifica JWT
3     return payload
4
5 def get_current_teacher(current_user: dict = Depends(get_current_user)) -> dict:
6     if current_user.get("role") != "teacher":
7         raise HTTPException(status_code=403, detail="Acceso restringido a docentes")
8     return current_user
9
10 def get_current_student(current_user: dict = Depends(get_current_user)) -> dict:
11     if current_user.get("role") != "student":
12         raise HTTPException(status_code=403, detail="Acceso restringido a estudiantes")
13     return current_user
```

*Mostrar en pantalla — Validación de acceso por roles (middleware) · archivo:
04_codigo_backend/06_middleware_autenticacion.png*

Términos clave para dominar:

- **API REST:** forma estándar en que el frontend pide datos al backend por HTTP.
- **OMR:** Reconocimiento Óptico de Marcas: leer las burbujas rellenas.
- **JWT:** credencial digital firmada que identifica al usuario en cada petición.
- **WebSocket:** canal abierto que permite mensajes en tiempo real.

Cierre de tu parte:

- Así funciona el backend. Ahora [Integrante 3] les muestra cómo se ve todo esto para el usuario.

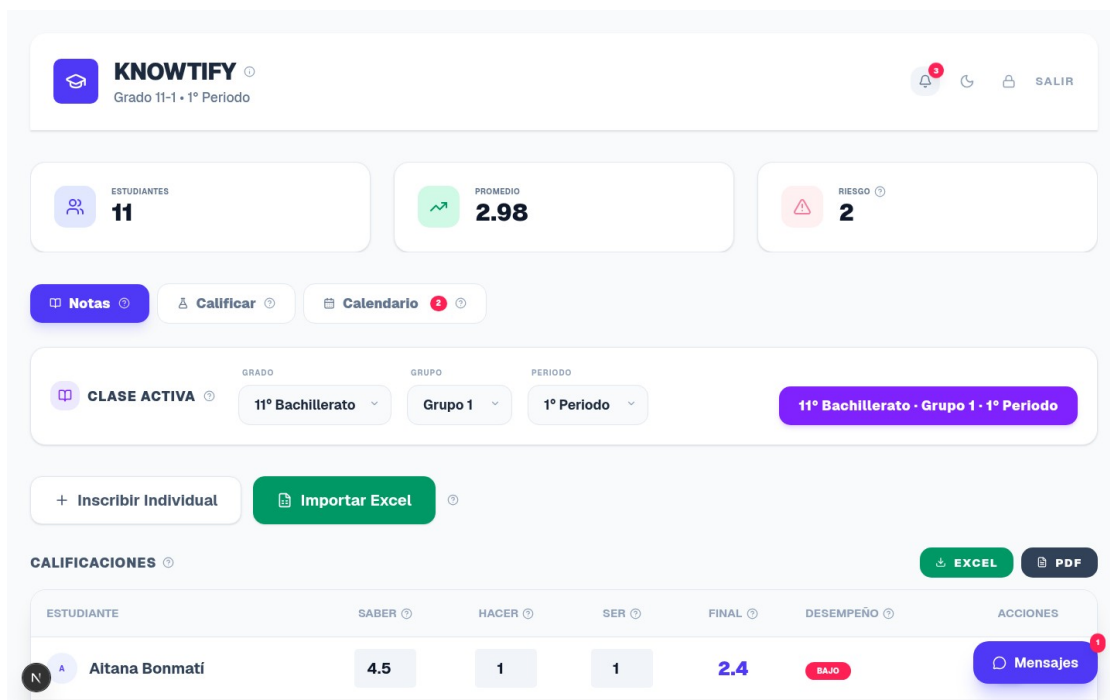
Integrante 3 — Frontend

Tiempo sugerido: 3-4 minutos. Eres quien hace la DEMO visual.

1. Stack y portal del docente

Qué decir:

- El frontend es lo que ve el usuario. Lo hicimos con Next.js 16, React 19 y Tailwind CSS.
- El portal del docente tiene la planilla de notas con un sistema de alertas de color: verde cuando va bien, rojo cuando hay riesgo. Se puede editar y exportar a Excel y PDF.

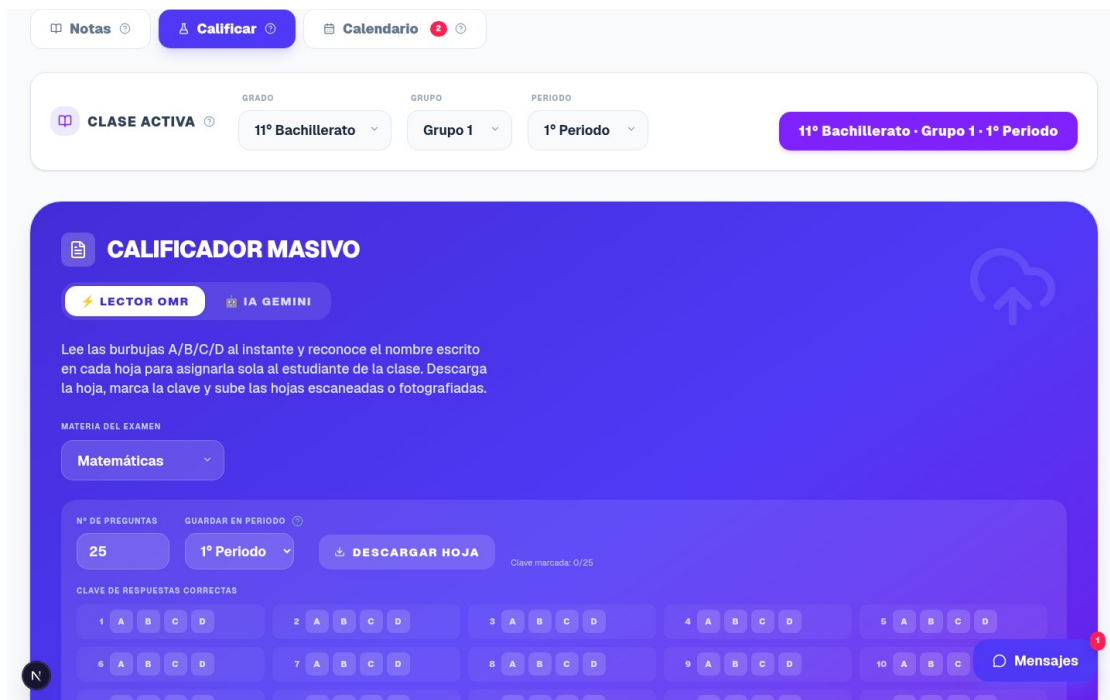


Mostrar en pantalla — Planilla del docente con alertas de color · archivo: 02_portal_docente/01_notas.png

2. Calificador y generador con IA

Qué decir:

- Desde aquí el docente sube las hojas escaneadas o una foto, y el sistema las califica solo, identificando al estudiante por su nombre.
- También puede generar un examen con IA escribiendo solo el tema, y reutilizar la clave en el calificador.

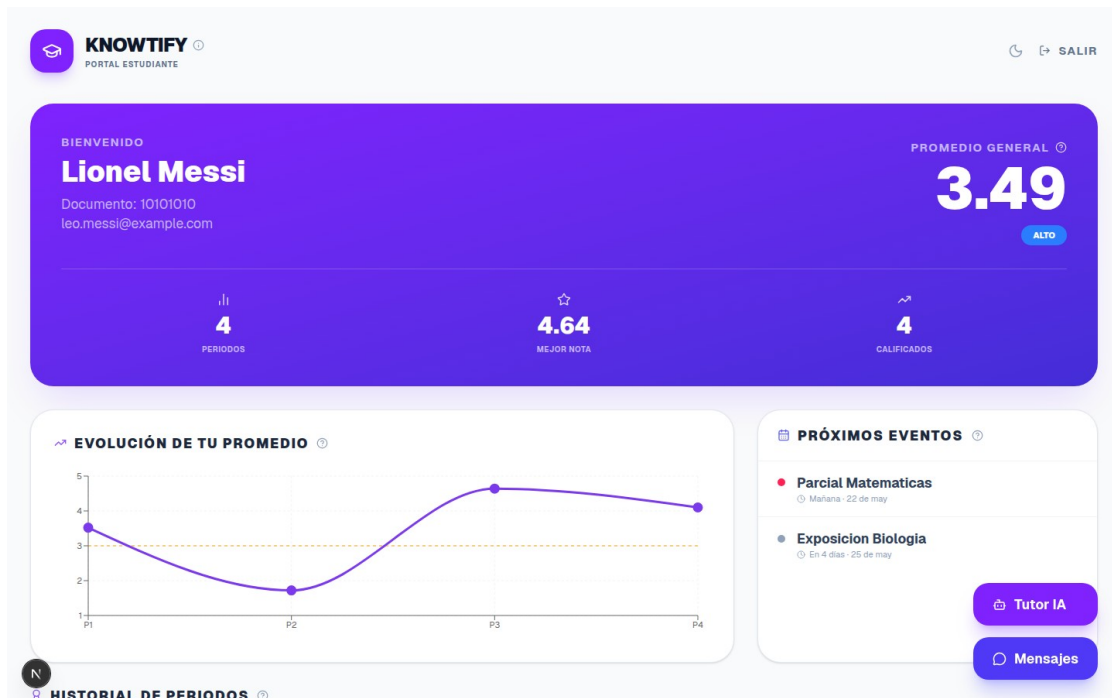


Mostrar en pantalla — Calificador masivo (Lector OMR / IA) · archivo: 02_portal_docente/03_calificar_omr.png

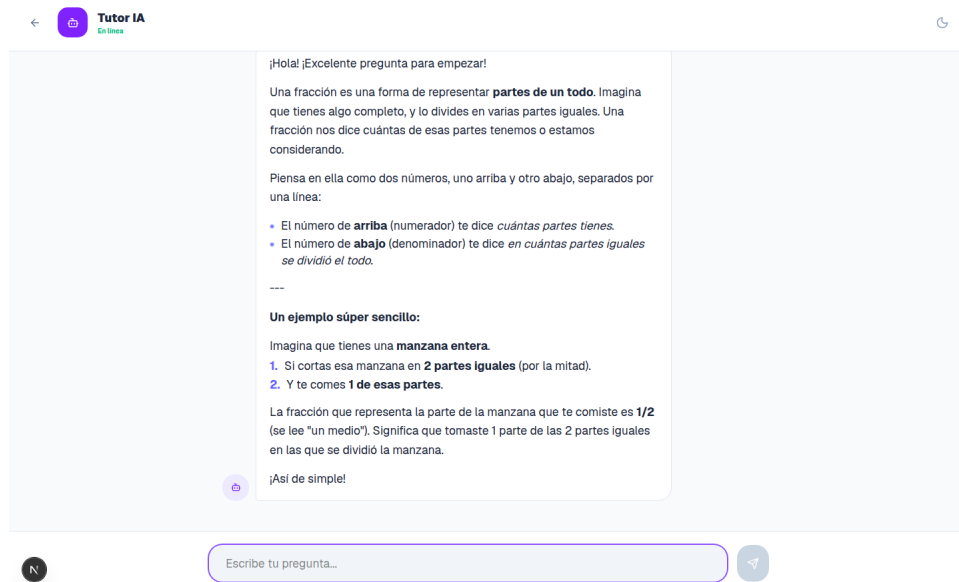
3. Portal del estudiante y Tutor IA

Qué decir:

- El estudiante ve su promedio general, una gráfica con la evolución de sus notas por periodo y los próximos eventos del calendario.
- Tiene también un Tutor IA disponible 24/7 que le explica temas con ejemplos, y mensajería directa con el docente.



Mostrar en pantalla — Tablero del estudiante (evolución y eventos) · archivo: 03_portal_estudiante/01_tablero.png



Mostrar en pantalla — Tutor IA respondiendo con formato claro · archivo: 03_portal_estudiante/06_tutor_ia.png

4. Diseño responsive y modo oscuro

Qué decir:

- Todo está diseñado para funcionar en el celular y tiene modo oscuro para cuidar la vista.

- Y para que el código sea ordenado, toda la comunicación con el backend pasa por un único cliente que añade el token y maneja los errores en un solo lugar.
- (Si hay tiempo, mostrar la versión móvil:
03_portal_estudiante/03_tablero_movil.png)

Cierre de la exposición:

- En resumen: KNOWTIFY le ahorra tiempo al docente y le da al estudiante una visión clara de su desempeño. ¿Tienen alguna pregunta?

Posibles preguntas del jurado (para los tres)

P: ¿Qué pasa si la IA no reconoce bien el nombre de un estudiante?

R: La hoja queda marcada en amarillo y el docente la asigna manualmente desde una lista. No se pierde ninguna hoja.

P: ¿Necesita internet para calificar?

R: La lectura de las burbujas (OMR) es local y no necesita internet. Solo se usa internet para la IA (identificar el nombre o generar exámenes).

P: ¿Cómo protegen los datos y las notas?

R: Con autenticación por token JWT, contraseñas cifradas con bcrypt, validación por roles y límite de intentos de inicio de sesión.

P: ¿Por qué eligieron leer el nombre en vez del número de documento?

R: Para ahorrar tiempo: el docente no tiene que escribir ni buscar nada; el sistema asigna cada hoja sola. La IA elige de la lista cerrada de la clase, lo que la hace fiable.

P: ¿La plataforma funciona en celular?

R: Sí, el diseño es responsive (se adapta a pantallas pequeñas) y tiene modo claro y oscuro.

P: ¿Está desplegada en internet?

R: Está preparada para desplegarse en la nube (Render) mediante un archivo de configuración; las pruebas se hicieron en entorno local.

P: ¿Qué tecnologías usaron?

R: Backend: FastAPI, PostgreSQL, OpenCV y Gemini. Frontend: Next.js, React y Tailwind CSS.

— Éxitos en la exposición —